Identifying and Defining

- **Identify the level of complexity you will aim to achieve**

For this project, I will be aiming to achieve a complexity level of 4, with a modifier of 110%. This will include at least five different fonts, as well as precise handwritten characters, which will vary in noise, weight, style, and can be rotated by up to 20 degrees. I have selected this level as I believe that it will present a meaningful challenge, while still being achievable within the project timeframe.

- **Create a set of functionality requirements, clearly defining what the program will be able to do**

The program will have a menu page, which will allow the user to choose between training mode and test mode.

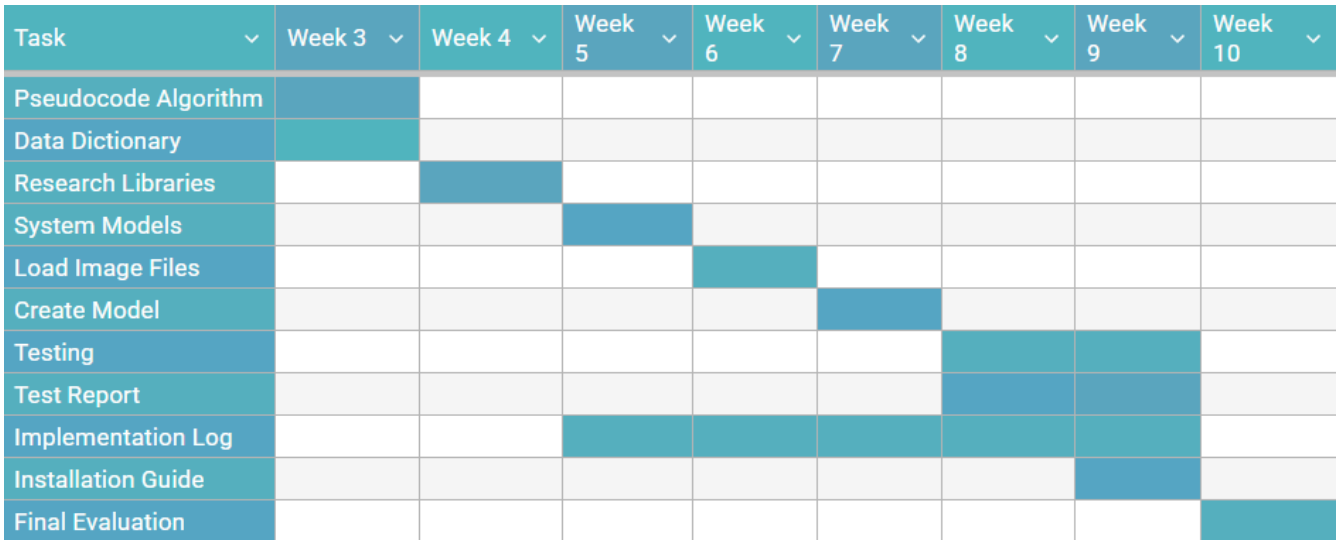
In Training Mode:

- The program should allow the user to load a dataset
- It should then train a machine learning model using these images
- It should save this trained model to a file in an appropriate format
- It should send a message when the model has been trained and allow the user to go back to the main menu

In Testing Mode:

- The program should be able to load the model which was previously saved in training mode
- It should allow the user to insert a file/folder of untested images
- It should then preprocess this image and use the trained model to predict the character for each file
- Output the prediction as well as its confidence score for that prediction
- It should send a message when all files have been tested which notifies the user and allows them to renavigate to the menu.

- **Create a Gantt chart for the project**



Research and Planning

- **Write a pseudocode algorithm for the K-Nearest Neighbour (KNN) method of machine learning. You may base this on existing algorithms or code, but must ensure it uses NESA-style pseudocode. You may assume that you have access to a function, euclidean_distance, which is passed two points and returns the Euclidian distance between them.**

```
None
BEGIN knn (point, allPoints, k):
#Each point has the form [x, y, label] - expand dimensions as needed
        distances = [], nearest = []
        location1 = (point[0], point[1])
        FOR i=0 TO length of allPoints STEP 1
                location2 = (allPoints[i][0], allPoints[i][1])
                dist = EuclideanDistance(location1, location2)
                append [dist, allPoints[i][2] to distances
        NEXT i
```

```
        distances = sort(distances)
        FOR i=0 to k STEP 1
                append distances[i][1] to nearest
        NEXT i
        point[2] = mode(nearest)
        return point
END

BEGIN mode (array)
        #Find the value in array that appears most often
        record = {}
        maxValue = 0
        mode1 = ''

        FOR i=0 TO length of array STEP 1
                record.array[i]++
                IF record.array[i] > maxValue THEN
                        maxValue = record.array[i]
                        mode = array[i]
                ENDIF
        NEXT i
        return mode
END

BEGIN sort(array)

    IF length(array) <= 1 THEN
        RETURN array
    ENDIF

    pivot = array[length(array) - 1]
    high, low = [], []

    FOR i=0 TO length(array) - 1 STEP 1:
        IF array[i] <= pivot THEN
            APPEND(low, array[i])
        ELSE
            APPEND(high,array[i])
        ENDIF
    NEXT i

    sorted_low = sort(low)
    sorted_high = sort(high)

    RETURN sorted_low + [pivot] + sorted_high

END
```

- **Create a data dictionary for your algorithm with columns for Name, Type, Storage Size, Description, Validation. Ensure that data structures are specifically identified and expanded if necessary. Only specify Validation when appropriate.**

Name	Type	Scope	Storage Size	Description	Validation
knn	Function	-	-	Passed a new point and all the existing points, and a value for k, calculates the likely category for the new point, and returns the new point.	-
sort	Function	-	-	Sorts an array of [distance, label] values in ascending order using quicksort	-
mode	Function	-	-	Returns the most frequent label in an array	-
k	Integer	knn	32 bits (4 bytes)	The number of nearby points to measure	> 0
dist	Float	knn	64 bits (8 bytes)	The calculated euclidean distance between 2 points	>=0
x	Integer	point	32 bits (4 bytes)	The x coordinate for a point	>= 0
y	Integer	point	32 bits (4 bytes)	The y coordinate for a point	>= 0
label	String	point	-	The category (character) of a point	-
point	List of x,y,label	knn	-	The details about a new point added to the data set	= 3
allPoints	Array of points	knn	-	A full dataset of all known points	>= k
distances	Array of lists of dist,label	knn, sort	-	Stores calculated distances with associated labels in ascending order	-
location1	Tuple of integers	knn	64 bits (8 bytes)	The x and y coordinates of the new point	-
location2	Tuple of integers	knn	64 bits (8 bytes)	The x and y coordinates of the current existing point	-
nearest	list of label	knn, mode	-	The k closest labels used to determine the new point's category	= k
record	dictionary	mode	-	keeps track of how many times each label appears	-
maxValue	Integer	mode	32 bits (4 bytes)	records the highest frequency found for any label	-
mode1	String	mode	-	records the most frequent label	-

- **Research five libraries that could be used in the intended project. For each, briefly summarise the purpose, requirements, and ease of use.**

1. **NumPy**

NumPy is a library used for numerical computing which provides fast mathematical operations such as array slicing, reshaping, and broadcasting. For this project, this library can be used to handle and manipulate image data by representing images as multidimensional arrays. It can be installed through the command:

```
None
pip install numpy
```

,and it's only prerequisite is having python installed, as different versions of NumPy are compatible with different versions of python. The numerical operations are completed using a C-based backend (e.g. OpenBLAS), which is usually automatically installed with the NumPy. Numpy is extremely beginner friendly as it uses expressive syntax and extensive documentation to guide users.

2. **Scikit-learn**

Scikit-learn is a library used for a wide range of machine learning tasks such as classification and regression. This includes algorithms such as KNN, which is directly used in this OCR project, allowing for easier implementation and testing. It is installed through the command:

```
None
pip install scikit-learn
```

,and the latest version (1.7) only supports Python 3.10 or newer. Sci-kit learn has a wide range of dependencies, such as numpy, scipy, and matplotlib. It is designed to be

very easy to use with a user-friendly interface and clear documentation which includes tutorials

3. TensorFlow

Tensorflow is a library which is primarily used for machine learning and artificial intelligence. This includes building and training deep learning models for OCR, such as CNNs, which are ideal for typed and handwritten characters. This can be installed by

```
None
pip install tensorflow  //for a CPU-only installation,
or
pip install tensorflow[and-cuda] //for GPU users
```

Tensorflow supports Python versions 3.9 to 3.12, and installs many packages, such as keras, bazel, and numpy. Tensorflow is considered to be difficult to learn and use with a large amount of programming skill needed, as even though it is more powerful and accurate than alternatives, it is very complex and not recommended for beginners. However, keras simplifies this by providing a high-level API which is intuitive and easy to use, reducing its difficulty for beginners.

4. PyTorch

PyTorch is a machine learning library used for building and training deep learning models such as CNNs. This is useful for advanced OCR tasks such as handwriting recognition, as this library contains a dynamic computational graph which allows developers to easily change their model and structure, increasing flexibility when testing and debugging. The latest version of PyTorch for Windows requires Windows 7 or greater and Python 3.9-3.12. To install for CPU only, the command required is

```
None
pip install torch --extra-index-url https://download.pytorch.org/whl/cpu
```

While the GPU enabled version is:

```
None
pip3 install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu117
```

However, GPU acceleration requires an eligible Nvidia driver with a CUDA toolkit. Pytorch has moderate difficulty for beginners, as while it has more flexibility and has easier debugging than Tensorflow, it requires more setup than a library such as scikit-learn, as it is more comprehensive.

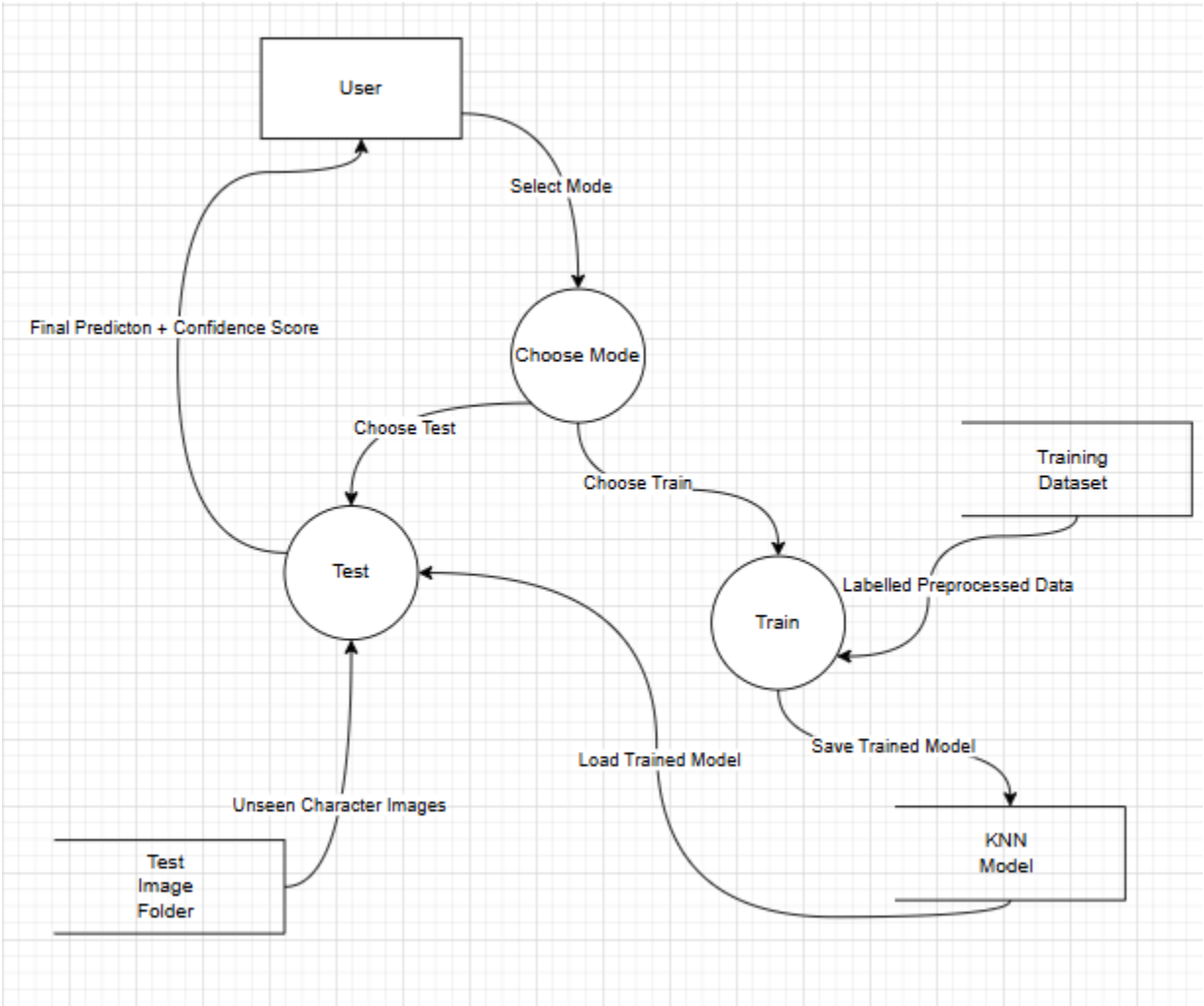
5. Pillow

Pillow is a powerful library used for image loading and preprocessing. While it doesn't have any OCR capabilities, it can be used for this project by resizing characters (to 28x28), converting images to grayscale or binary, rotating them, and applying filters to clean up image noise. This can simply be installed through:

```
None
pip install pillow
```

, with the newest version supporting Python 3.9 to 3.13. Pillow is very easy to use as it has an intuitive API which makes image manipulation tasks very accessible.

- **Create a level 1 data flow diagram based on the [project requirements](#).**



Producing and Implementing

- **Describe the backup and version control processes you will utilise, justifying the timing and location/s.**

To ensure that my project is safely stored and that all changes are tracked over time, I will be using a combination of manual backups and version control practices. My project folder will be stored in OneDrive, which provides cloud-based backup and version history. This will automatically back up all my files as I edit them. This will also save previous versions in case I need to revert back to them. Furthermore, this will allow me to work on the project on both my home computer and school laptop. After every session, I will also manually save a copy of my project folder, which will be beneficial in case of cloud-failure. These copies will be appropriately named so that I can easily differentiate between them and not get them mixed up.

- **Record all progress in an implementation log. For each aspect of the program, you must clearly describe:**
    - **What this component is meant to achieve and how you intend to implement it**
    - **Any references or sources that are guiding you (with links)**
    - **How you have deviated from the reference or source described**
    - **Issues that arose during the implementation (including both errors and struggles to understand the reference/source)**
    - **How you solved the issue/s (include links to additional sources if necessary)**

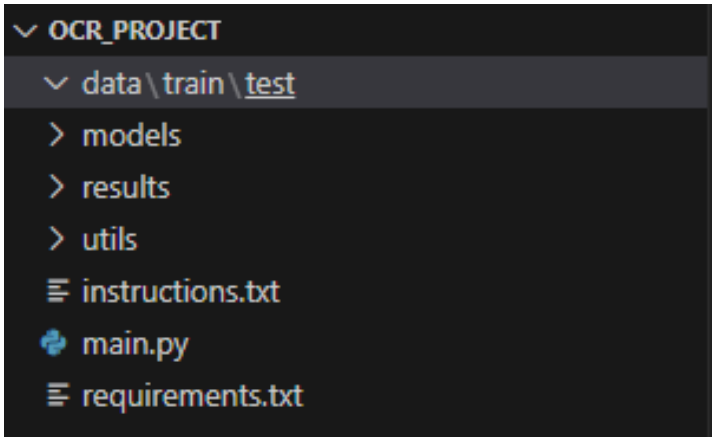
  You may support your descriptions with screenshots of:

    - **Your program's functionality**
    - **Errors (either incorrect output or error messages)**
    - **Research, tutorials, sources**

Project Setup

Before I begin working on the actual project, I decided to set up a clean and organised workplace in order to set the foundation for my project. This includes creating my project folder and files as well as installing required libraries. To understand what this should look like, I asked ChatGPT for assistance.

To create the project folder, it utilised the terminal, however as I didn't understand how this worked, I decided to create it manually.



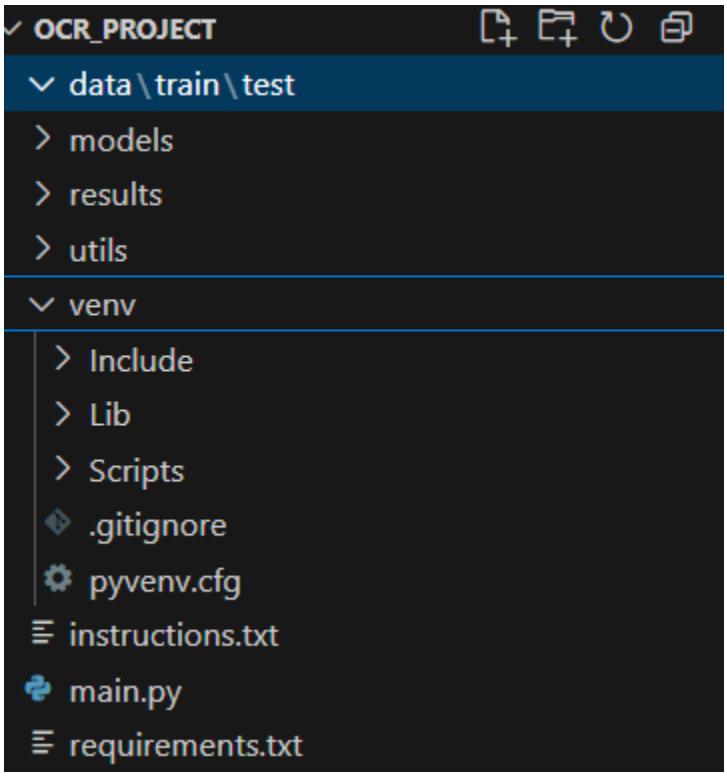
Next, ChatGPT said to create a virtual environment, which is a self-contained folder which holds its own installation of all libraries. This is necessary as it prevents any conflicts due to having multiple versions of the same library installed.

To do this, I found a tutorial on W3 schools ([Python Virtual Environment - venv](#)) which seemed pretty straightforward. However, as I was implementing it, I ran

```
>>> venv\Scripts\activate
  File "<python-input-0>", line 1
    venv\Scripts\activate
         ^
SyntaxError: unexpected character after line continuation character
```

into this repeated error.

I put this error message into ChatGPT and it pointed out that this was because I was running the command inside the python interpreter and not in the terminal. This was a simple fix as all I needed to do was exit the python shell to fix this.



Next I needed to install required libraries. As I don't know all the libraries that I will be using as of yet, I decided to start with numpy and pillow as I need them for image preprocessing. I already knew how to download these as I had researched them as part of my research and planning phase. This was a simple process

Image Preprocessing

The first component of my project that I will be working on is Image Preprocessing. The goal of this is to load a character image and convert it into a standardised format so that my machine learning model can understand it. This should be in grayscale and sized at 28x28 pixels, which will then be converted into a numpy array.

Before programming, I downloaded the provided image files and added them to my project folder. I also created a file called preprocess.py under my 'utils' folder, which will contain my code for this component.

Firstly, I imported pillow and numpy. Next, I wanted to use the Pillow library to open the image, convert it to grayscale and ensure that it is sized at 28x28 pixels, and then convert it into a NumPy array. For this, I found a tutorial which showed how to convert an image into a numpy array. (How to Convert images to NumPy array? - GeeksforGeeks). I was also able to write the code for resizing the image and converting it to grayscale through the autofill option on Visual Studio Code. For grayscale, the code was '*Image.convert('L')*', which I didn't understand at first, but after a quick google search, I learnt that it stood for Luminance, which is just the brightness/intensity of the image and not relevant to the actual colour.

At this point, I also realised that I had to choose whether I was going to use KNN or CNN, as if I wanted to use KNN, I would need to flatten my array down to a 1d array. Although I initially wanted to challenge myself and aim for a high complexity level, I realised that it would be difficult to balance this project with all my other assessments, so I decided to opt for a simpler approach and use KNN. However, if I finish this project with time to spare, I may revisit this.

Since there are hundreds of training images that the program needs to process, I need to write another function which will look through the folder with the

images and extract the image path as well as the label (character) and preprocess all the images. To do this I used the function 'os.listdir()' as per [Python os.listdir() Method](). This method returns the list of all entries in the directory.

I used this in a for loop to get every file name, check if it is a png, and get the label, which is the last part of the image name in the provided files. To check if a file is an image, I discovered the function endswith() ([Python String endswith() Method]()) to check if the file ends with '.png'. To get the label name, I used the split function to get the last bit of the file name and replace '.png' with ''. This should result in a file called 'Garamond_bold_32_D.png' to return 'D' as the label.

Now that I had the image path, I ran the preprocess function to process the image and then appended this into a list along with its label.

To test if this worked, I asked ChatGPT to write a temporary function to test it. However, when I ran the program, I received no output at all. At first, I thought this was because I forgot to activate my venv, however after activating this, I still received no output.

After some research on virtual environments ([Virtualenv and venv: Python virtual environments explained | InfoWorld]()), I realised that I was supposed to run my script in the terminal rather than just visual studio code. I tried this and received the following error message:

```
--- Starting test of load_dataset function ---
Attempting to load data from: 'data/train/image_files'

--- AN ERROR OCCURRED DURING load_dataset EXECUTION ---
This is expected as we are testing for issues.
Error Type: FileNotFoundError
Error Message: [Errno 2] No such file or directory: 'Cookie_regular_32_0.png'
Traceback (most recent call last):
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\main.py", line 35, in main
    loaded_data_for_test = load_dataset(TRAINING_DATA_DIR)
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\utils\preprocess.py", line 23, in load_dataset
    processed = preprocess_image(image_path)
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\utils\preprocess.py", line 9, in preprocess_image
    img = Image.open(image_path) #uses pillow to open the image file
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\venv\Lib\site-packages\PIL\Image.py", line 3505, in open
    fp = builtins.open(filename, "rb")
FileNotFoundError: [Errno 2] No such file or directory: 'Cookie_regular_32_0.png'
----------------------------------------
```

This error states that my image files can not be found. This is due to the fact the image path is currently just the file names rather than the whole trajectory. This was an easy fix as all I needed to do was combine the file name with the dataset path. I ran the program again but this time received a different error:

```
--- AN ERROR OCCURRED DURING load_dataset EXECUTION ---
This is expected as we are testing for issues.
Error Type: AttributeError
Error Message: module 'PIL.Image' has no attribute 'convert'
Traceback (most recent call last):
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\main.py", line 35, in main
    loaded_data_for_test = load_dataset(TRAINING_DATA_DIR)
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\utils\preprocess.py", line 23, in load_dataset
    processed = preprocess_image(image_path)
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\utils\preprocess.py", line 10, in preprocess_image
    img = Image.convert("L") #converts image to grayscale
          ^^^^^^^^^^^^^
AttributeError: module 'PIL.Image' has no attribute 'convert'
```

This time, the error was because I called the reize() and convert() methods on the ‘Image’ module rather than the actual object, which was a simple fix.

I ran the program again and this time had no issues, meaning that my program ran successfully.

Model Training - KNN

For this component, I was working on a different device. When I tried to run my existing program, I got this error.

```
did not find executable at 'C:\Users\Humza\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe':
 The system cannot find the path specified.
```

This was because my virtual environment which was created on my other device uses a python installation which doesn't exist on this device. I asked ChatGPT how to fix this, but I couldn't really understand it so I decided to just delete the virtual environment entirely and install everything globally.

Now that that was done, I began working on the next phase, which is using this processed dataset to train a knn model. This is the tutorial that I will be referencing:

Sklearn KNeighborsClassifier (in Python) - K-Nearest Neighbors in Scikit-l...

Firstly, I downloaded Scikit learn and imported it into my project

Next, I had to identify the independent and dependent variables from my dataset. Since the tutorial used a preprocessed dataset, I couldn't use it for this step. I set the independent variable as the image data, as that is what the model will be using to make its predictions, and the dependent variable as the label, as this is what the model will be trying to predict. This was quite simple as I just had to set the independent variable as the first item in my processed numpy array (the image data) and the dependant variable as the second item:

```
x = np.array([item[0] for item in dataset])
y = np.array([item[1] for item in dataset])
```

The next step in the tutorial was data preparation and splitting using a procedure called 'train_test_split'. The purpose of this was to split the data into

a training set and a testing set to see how the model performs on new data. However, as I currently only wanted to train the data and I had already preprocessed it, I decided to skip this step.

Next, I created the knn model and set k to 5 as a starting point, meaning that the model will look at the 5 closest images when predicting a new one. I will most likely change this later. This was done by simply importing the 'KneighborsClassifier' from scikit-learn and letting nearest neighbours = 5.

The next step was to train the model using the .fit() method. This stores the training data and their associated labels, however it doesn't actually perform any calculations as KNN is a 'lazy' learner.

The next step had to do with the testing component for this project and the actual predictions, so I stopped following the tutorial at this point.

I thought I had finished the training component at this point, however after referring to the assessment notification, I realised that I still had to save these model values for later use. I researched how to do this and came across a forum which suggested using pickle (How to save a knn model? - Data Science Stack Exchange). This is a built-in python module which can save/serialize any python object as well as load/deserialize it later without the need for retraining.

From there, I found this article which showed how to save a model to a string using pickle (Saving a machine learning Model - GeeksforGeeks). To implement this, all you need to do is use the pickle.dumps() function to save the model as a variable by converting it into a byte string, and it can later be converted back into a model to make predictions.

While researching how this worked, I learnt that this only serializes the object but doesn't actually save this. This means that this will disappear when the program is closed and will need to be retrained when opened again. To fix this issue, I need to save the trained model to a file so that it can be used later. I

asked ChatGPT for assistance with this and it said to implement:

```
with open("models/knn_model.pkl", "wb") as f:
    pickle.dump(knn, f)
```

This opens a file called knn_model.pkl in my models folder, where the 'wb' stands for write binary, as pickle only works with binary data.

This was the final step to completing training my model. This process was a lot simpler than I expected, as I encountered minimal errors during this.

I tried running my train file to save my trained dataset, however I received this error:

```
C:\Users\Humza\OneDrive\Desktop\ocr_project>python train.py
python: can't open file 'C:\\Users\\Humza\\OneDrive\\Desktop\\ocr_project\\train
```

This is because my train.py file is in a subfolder and not my main project folder. To fix this, I went to my train folder and ran the program from there, however I encountered another issue:

```
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\data\tra
    from utils.preprocess import load_dataset
ModuleNotFoundError: No module named 'utils'
```

This was because the script is running from a subfolder which won't check folders above its level. To resolve this, I decided to just delete my train subfolder and have the train.py file as well as my image files in my main project folder. I also wrote a temporary print statement to tell me if the model has been created.

This fixed the issue and created the knn_model.pkl file in my models folder:

```
C:\Users\Humza\OneDrive\Desktop\ocr_project>python train.py
model trained and saved
```

```
models
  knn_model.pkl
```

Testing

Before beginning testing, I realised that I needed a different dataset of images to test the accuracy of my model. After consulting with one of my classmates, I decided to split my existing dataset into a training and testing set, which I will implement through the 'train test split' function which came up earlier. This allocates a certain proportion of the image files which will be used for testing and the rest will be used for training. To implement this, I referred back to the original tutorial. All I had to do was import the train_test_split function from scikit-learn and outline what proportion of my data was to be used for testing. For now, I decided to use 0.3 (or 30%)

The tutorial also defined the 'random state' at 42. Ther tutorial didn't really explain this so I had to research what this meant. ([What is "random-state" in sklearn.model_selection.train_test_split example? - Stack Overflow])

I learnt that setting the random state at an integer value fixes the amount of randomness and ensures that the same split is used each time. This is important as if the split was different, the accuracy would vary by each test, making the result unreliable. The number 42 isn't actually necessary but apparently an inside joke in the coding community as 42 is the answer to life in The Hitchhiker's Guide to the Galaxy. This is what I have.

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=1)
```

I also needed to adjust the fit() method to use the training values for x and y:

```
knn_model.fit(x_train, y_train)
```

Now I can move on to my testing function.

For this component, I will have to load the model from my saved pickle file, load and preprocess the test images, and use my model to predict the images. It should then display the final guess, confidence score, and what the actual label was.

Firstly, I loaded in my saved knn model, which was a similar process to saving it in the first place. The first line was almost identical to before, except I had to change the 'wb' into 'rb', as now, my program is reading ('r') the file rather than actually writing it ('w').

```
with open("models/knn_model.pkl", "rb") as f:
```

Then, I simply loaded in the pickle file which reconstructs the saved knn model using pickle.load(). For this step, I used the assistance of ChatGPT.

Since I was working on a new file for this, I had to import my dataset and once again split it into image data and labels. For this, I could just copy and paste my code from my train.py file.

A beginner's guide to building a KNN model: Everything You Need To Know

Using this tutorial, I used dataset.predict() to make my predictions. This runs the model on the test data and returns a list of predicted labels for each test image.

The next step is to display the confidence score for the prediction and the accuracy of the model. To calculate the confidence score, I came across this method called predict_proba(), which is a part of scikit- learn. I did some research on this to understand how it works (Understanding the predict_proba() Function in Scikit-learn's SVC - GeeksforGeeks)

Essentially, this finds the k closest neighbours and checks the labels of those neighbors to calculate the probability of each label. I'm not a hundred percent sure how to implement this, so to start, I decided to replace my .predict() method with .predict_proba(). However, I received this error:

```
Traceback (most recent call last):
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\test.py", line 15, in <module>
    prediction = dataset.predict_proba(x_test) #predicts labels for the test data
                 ^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'list' object has no attribute 'predict_proba'
```

At first, I thought that this was because I had to import a separate library to use this function, so I referred back to the article and imported SVC from sci-kit learn (don't really know what this is) and ran the program again, however this error persisted.

I then realised that I had made a silly mistake and used the predict function on the dataset rather than the knn model.

```
prediction = dataset.predict_proba(x_test)
```
(what I had)

```
prediction = knn_model.predict_proba(x_test)
```
(what it should be)

I ran the program again and received this for my predictions:

```
Predictions: [[0.4 0.  0.  ... 0.  0.  0. ]
 [0.  0.2 0.  ... 0.  0.  0. ]
 [0.  0.  0.  ... 0.  0.  0.2]
 ...
 [0.  0.  0.  ... 0.  0.  0. ]
 [0.  0.  0.  ... 0.  0.  0. ]
 [0.  0.  0.  ... 0.  0.  0. ]]
```

Here, each list shows the probability of each label (e.g A, B, C).

If I just use regular .predict(), it will only show the most likely value:

```
Predictions: ['0' '4' 'Z' '3' 'hash' 'S' 'C' '7' 'S' '9' 'A' 'ampersand' 'minus' 'at'
 '6' 'E' '1' 'S' 'E' 'exclmark' 'F' '1' 'greaterthan' 'B' 'hash' 'I' 'J'
 'K' 'R' 'lessthan' '0' 'minus' 'A' 'C' '8' 'X' 'plus' '1' 'exclmark' 'V'
 'E' 'T' 'A' 'E' '0' 'X' 'G' '0' '0' '1' '2' '3' '4' '3' '6' '7' '8' '9'
 'A' 'ampersand' 'asterisk' 'U' 'H' 'E' 'D' 'S' 'E' 'exclmark' 'F' '6'
 'greaterthan' 'H' 'hash' '1' 'J' 'K' 'L' 'lessthan' 'M' 'minus' 'N' '0'
 'P' 'percent' 'plus' 'Q' 'quesmark' 'R' 'S' 'T' 'N' 'V' 'W' 'X' 'Y' 'Z'
 '0' '1' '2' '3' '4' '5' '6' '7' '8' '9' 'N' 'asterisk' 'minus' 'U' 'P'
 'E' 'J' 'S' 'E' 'exclmark' 'T' 'E' 'greaterthan' 'P' 'N' '1' 'J' '0' 'L'
 'minus' 'M' 'minus' 'N' '0' 'I' 'S' 'plus' 'Q' 'exclmark' 'R' 'S' 'I' 'N'
 '0' '1' 'X' 'Y' '1' '0' '1' '2' '3' '4' '3' '6' '7' '3' '9' 'S'
 'ampersand' 'minus' 'U' 'B' 'E' 'D' 'S' 'E' 'exclmark' 'F' '8'
 'greaterthan' 'H' 'U' 'I' 'J' 'K' 'exclmark' 'minus' 'M' 'minus' 'N' '0'
 'P' 'percent' 'plus' '9' 'exclmark' 'R' 'S' 'R' 'N' 'V' 'W' 'S' 'Y' 'Z'
 '0' '1' '2' '3' '4' '5' '6' '7' '8' '9' 'A' 'ampersand' 'asterisk' 'U'
 'H' 'C' 'D' 'dollar' 'E' 'exclmark' 'F' 'G' 'greaterthan' 'H' 'hash' 'I'
```

Since I want to show both the final prediction as well as the confidence score, I will need to somehow combine these two where .predict() shows the most likely prediction and .predict_proba() shows the probability of that prediction.

To do this, I will try to create a for loop which gets the final prediction using .predict() and then find the confidence score from the list from predict_proba().

First, I got the length of the array and set it as the range of my for loop. I then got each final prediction and it's confidence score through this:

```
for i in range(len(x_test)):
    final_prediction = prediction[i]
    confidence_score = knn_model.predict_proba([x_test[i]])
```

I wrote a couple of print statements to test my results but received this error:

```
Traceback (most recent call last):
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\test.py", line 21, in <module>
    print('confidence score:' + confidence_score)
          ~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~
numpy._core._exceptions._UFuncNoLoopError: ufunc 'add' did not contain a loop with signature matching types (dtype('<U17'), dtype('float64')) -> None
```

I think this is because I have to convert my confidence score output to a string. I fixed this and ran the program again.

```
final predicton:V
confidence score:[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
```

I received this output but for every single test image. While this shows the final prediction and the confidence score for each letter, I want the confidence score to only show the probability of the chosen prediction.

I was stuck on this for a while so I decided to ask ChatGPT for help. My mistake was that '*knn_model.predict_proba([x_test[i]]*' returns a whole 2d array for each image, while I thought that x_test[i] would mean it would only return the value from the final prediction.

For this, I have a list of all possible labels given by .classes_, such as this:

```
['A', 'B', 'C', 'D']
```

And by using predict_proba(), I get a list of probabilities, such as this:

```
[0.05, 0.15, 0.70, 0.10]
```

Since each probability matches up to the corresponding label (e.g. in this case, the probability for 'A' is 0.05), I can use the index to find the corresponding probability for each final prediction, as shown below.

```
Index → 0     1     2     3
Label → 'A'   'B'   'C'   'D'
Value → 5%    15%   70%   10%
```

I implemented this into my code and tried running it again, howeverI received this error:

```
Traceback (most recent call last):
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\test.py", line 21, in <module>
    class_index = (knn_model.classes_).index(final_prediction) #get the index for the prediction
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'numpy.ndarray' object has no attribute 'index'
```

This is because knn_model.classes_ is a numpy array, and hence, I can't use indexing for it. This was a simple fix as I just had to convert it to a list. I tried

```
final predicton:0
confidence score:[0.4 0.  0.  0.  0.  0.  0.  0.  0.  0.2 0.  0.  0.  0.  0.  0.  0.  0.
 0.  0.  0.  0.  0.  0.  0.4 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0. ]
Traceback (most recent call last):
  File "C:\Users\Humza\OneDrive\Desktop\ocr_project\test.py", line 22, in <module>
    confidence_score = probabilities[class_index] #get the confidence score for the prediction
                       ~~~~~~~~~~~~~~^^^^^^^^^^^^
IndexError: index 4 is out of bounds for axis 0 with size 1
```

running it again and ran into this error:

The issue here is since predict_proba gives a 2d array with every image, when the index is 2, it gets the second row rather than the second value from the inner list. As seen in the above output, this is why the confidence score gave a whole list and also why I received the error. To fix this, I just had to make sure that I only extracted the first row, which was quite simple. I ran the program again and received the expected output, where it gives the final prediction and its confidence score:

```
final predicton:N
confidence score:0.8
final predicton:0
confidence score:0.6
final predicton:P
confidence score:1.0
final predicton:percent
confidence score:0.4
final predicton:plus
confidence score:1.0
final predicton:Q
confidence score:0.8
final predicton:quesmark
confidence score:0.8
final predicton:R
confidence score:1.0
final predicton:S
confidence score:0.6
```

Increasing amount of data + Improving accuracy:

Now that my knn model is functional, I want to increase its accuracy. I'm going to do this by first increasing the amount of training data. I will be doing this by generating new images using the script provided.

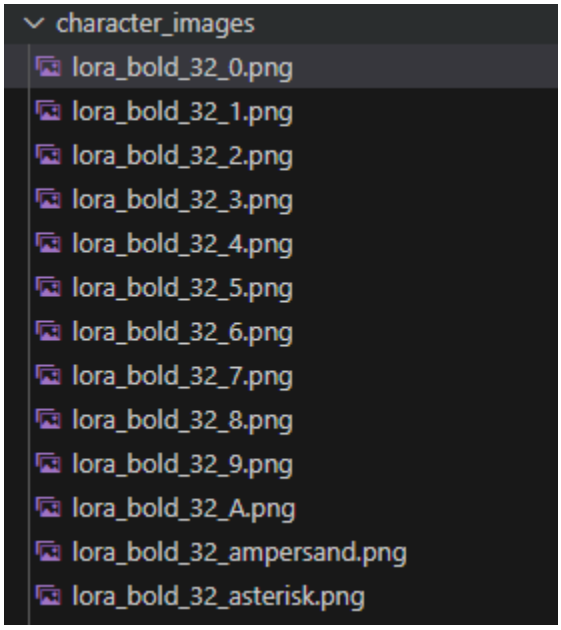
Firstly, I created a new python file and pasted the script into it. I also deleted all my existing training data to restart.

Next, I chose my font families:

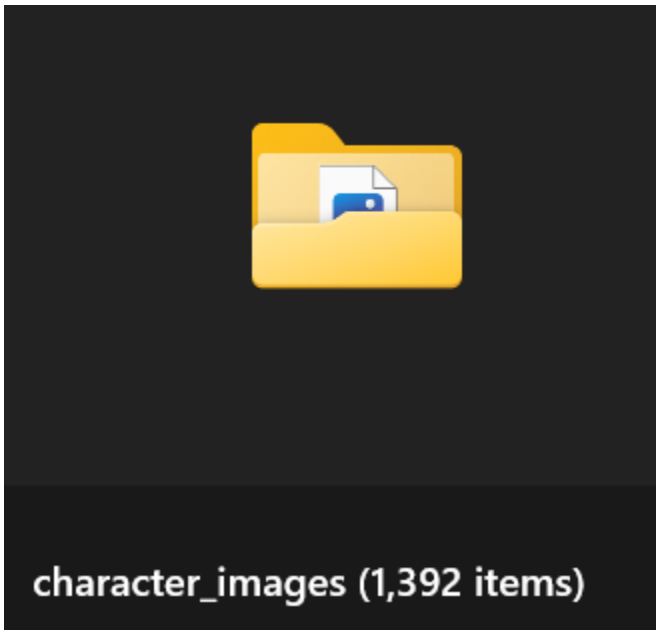
1. Roboto (san-serif)
2. Open Sans (san-serif)
3. Lora (serif)
4. Source Serif 4 (serif)
5. Roboto Mono (monospace)

I then downloaded 5 different ttf files from each family in order to ensure that my model is trained on different weights and styles, as per the marking criteria.

After downloading the fonts, I tried running the script for one of them. This successfully generated images:



I now repeated this process for all the fonts. By the end, I had 1392 image files.



Now that this was done, I ran my [train.py]() function again to retrain all my data

After this was done, I tested my model again:

```
confidence score:20%
final predicton:C
confidence score:40%
final predicton:D
confidence score:40%
final predicton:dollar
confidence score:40%
final predicton:E
confidence score:60%
final predicton:exclmark
confidence score:100%
final predicton:F
confidence score:80%
```

I think the reason that all the confidence scores are multiples of 20 is because my k value is set at 5, meaning that the confidence score is how many of the final predicted labels there are in the 5 closest neighbors. After searching up what my k value should be, I discovered that the general rule of thumb is the square root of the total number of training data. Since I had around 1400 images, and 30% of this was testing data, this meant that I had around 1000 images of training data, so my k value should be 31.

I retrained my data for this and then ran [test.py]() once again.

```
final predicton:S
confidence score:61%
final predicton:T
confidence score:51%
final predicton:U
confidence score:48%
final predicton:V
confidence score:64%
final predicton:W
confidence score:58%
final predicton:X
confidence score:61%
final predicton:Y
confidence score:64%
final predicton:Z
confidence score:51%
```

I now had more distinct probability values, however the predictions weren't great. After a bit of research ([KNeighborsClassifier — scikit-learn 1.7.0 documentation]()), I found out that I can weight each neighbour by how close they are. This means that closer neighbours are considered more than further away neighbours, which can help improve the accuracy of my model. This was very easy to implement, as I just had to set the weight to distance when creating the model:

```
knn_model = KNeighborsClassifier(n_neighbors=31, weights='distance')
```

I ran the program again, and this time it was much more accurate:

```
final predicton:R
confidence score:100%
final predicton:S
confidence score:100%
final predicton:T
confidence score:99%
final predicton:U
confidence score:100%
final predicton:V
confidence score:100%
final predicton:W
confidence score:100%
final predicton:X
confidence score:67%
final predicton:Y
confidence score:100%
final predicton:Z
confidence score:100%
```

I've just realised that I've been looking at the overall confidence score and not the actual testing accuracy, which shows how confident the model is and not whether it is correct or not.

To calculate the overall accuracy score, I found out that there is a library part of sci-kit learn which simplifies this process.

[K-Nearest Neighbors (KNN) Classification with scikit-learn | DataCamp]

After looking at this website, I realised that this was very easy to implement, as I just used the accuracy_score function which compares the predicted labels with the actual labels. After adding this to my program, I ran it again and received this accuracy score:

```
Overall testing accuracy: 95.76%
```

This was surprisingly high!

I was looking back at the task notification and realised that for complexity level 4, I also needed to include neat handwritten characters as well as rotation of up to 20 degrees. For the handwritten characters, I was allowed to use fonts that look handwritten.

To find these, I went to google fonts and found 2 fonts which looked appropriate.

For the rotation, after asking ChatGPT, I learnt that I could do this by creating a list of different angles, and then creating a for loop which creates multiple iterations of each character, one for every angle in the list.

This can be implemented by using the rotate() function included in the pillow library.

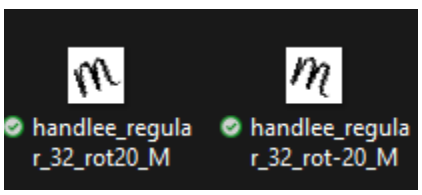
([Python PIL | paste() and rotate() method - GeeksforGeeks])

I decided to do the rotation in increments of 10, as to avoid too much training data which can lead to inefficiency and my model running very slowly. This will most likely limit the accuracy, however I feel like it's the best decision.

```
rotation_angles = [-20, -10, 0, 10, 20]
```

Since I had to reuse the same logic for the character and symbol generation, I decided to create a helper function which will reuse the same logic without duplicating code, improving the readability of code. This part was mainly inspired by ChatGPT. However, I realised that I also had to change the format in which file names are changed, in order to avoid any duplicate files which could mess up my program. To do this, I just edited the .save() function to include the rotation. I also moved it to the helper function as this process had to be repeated for the characters and the symbols.

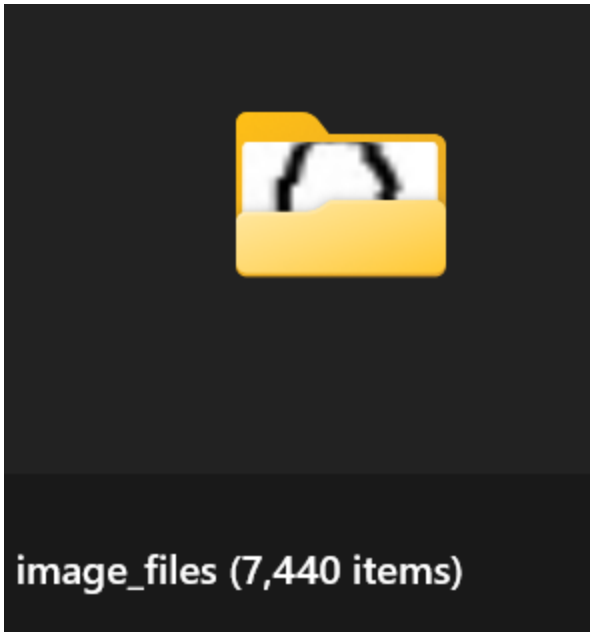
Now that this was done, I realised that I had to re-generate all my images to include the rotations, meaning I had to delete them first. After doing this, I tested the program by running it on one of my new handwritten fonts. While this successfully created multiple rotated iterations of each letter:



It didn't do this for the symbols. After referring back to my code, I realised that this was because at the end of my symbol generation function, I call letter (which doesn't exist in this function) instead of symword. After this change, my program was working properly.

I manually did this for all fonts.

After this process was completed again, I had over 7000 image files of training data.



Since 70% of this is training data (4900ish files), I found the square root of this and hence, set the number of neighbours to 70.

This recorded an overall testing accuracy of:

```
Overall testing accuracy: 94.49%
```

While this is slightly lower than before, this test involved neatly handwritten images as well as rotations of up to 20 degrees, which is trickier. Overall, I am happy with this.

Finding the best k value

While I have been operating under the assumption that the best k value is the square root of the total training data, this is not always the case. When researching how to get the most accurate k value, I came across something called Knn hyperparameter tuning.

This uses a function from sci-kit learn called GridSearchCV, which tests every combination of parameters to find the most accurate one.

https://dev.to/anurag629/gridsearchcv-in-scikit-learn-a-comprehensive-guide-2a72

While this tutorial wasn't fully applicable to my project as it didn't use knn, it gave me a good idea of how gridsearchcv worked.

Firstly, I needed to define my hyperparameters. Since I had only used the number of neighbours and the weight so far, I decided to stick with this for now.

For the number of neighbours, I created a list which contained all numbers from 1 - 99, as I felt that anything over that was excessive.

```
'n_neighbors': list(range(1, 99)),
```

For the weights, I just set it to either uniform (the default option) or distance (what I was currently using).

Next, I used the GridSearchCV function, defining the parameter grid and ensuring that it is scoring based on accuracy.

After that, I just had to use the .fit() method to test all the parameters, and then create a knn model which uses the most accurate ones.

The tutorial only showed how to print the best parameters, but I wanted it to see if it was possible to automatically use the best parameters instead of finding them through gridsearch and manually changing them.

https://stackoverflow.com/questions/30102973/how-to-get-best-estimator-on-gridsearchcv-random-forest-classifier-scikit

I came across this post, which once again wasn't using knn. However, I saw that they used a method called .best_estiomator_, so I decided to try it out.

I implemented this and tried running my code, and received this warning message:

```
C:\Users\Humza\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\P
ython313\site-packages\sklearn\utils\_response.py:203: UserWarning: The number of unique classes is greater than 50%
 of the number of samples.
  target_type = type_of_target(classes)
C:\Users\Humza\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\P
ython313\site-packages\sklearn\utils\_response.py:203: UserWarning: The number of unique classes is greater than 50%
 of the number of samples.
  target_type = type_of_target(classes)
C:\Users\Humza\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\P
ython313\site-packages\sklearn\utils\_response.py:203: UserWarning: The number of unique classes is greater than 50%
 of the number of samples.
  target_type = type_of_target(classes)
C:\Users\Humza\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\P
ython313\site-packages\sklearn\utils\_response.py:203: UserWarning: The number of unique classes is greater than 50%
 of the number of samples.
  target_type = type_of_target(classes)
```

This line was repeated thousands of times. What it basically was saying is that I don't have enough samples, which I found weird since I had over 7000 image files.

I decided to write a print statement to check the total number of samples that the program detects, as well as how many unique classes it detects.

```
samples: 7440
classes: 48
```

This received the correct output, meaning that I shouldn't be getting the warning.

[Unjustified "number of unique classes > 50%" warning in CalibratedClassifierCV · Issue #31583]

I found this post on github about someone experiencing the same issue, however there was no reason provided as to why.

I noticed that this wasn't actually an error message so I decided to test my program anyway. I received this accuracy, which was very high:

```
Overall testing accuracy: 98.56%
```

Since the program was working, I figured that I could just hide the error message, just like the person in the GitHub post did.

My accuracy number felt very high, so I decided to look back at my code. I realised that even though I had the train test split, I wasn't correctly using it, as my test.py script went through all of the available data. This has inflated the accuracy, as it has already been trained on 70% of this data.

To fix this, I ChatGPT why this could be happening and how this could be fixed. It pointed out that this was because in my test.oy file, I was loading the entire dataset again:

```
dataset = load_dataset("image_files")  # load the dataset from my image folder

x_test = np.array([item[0] for item in dataset]) #seperates image data and labels from the list of tuples
y_test = np.array([item[1] for item in dataset])
```

It suggested to save the test set model when training, like this:

```python
with open("models/x_test.pkl", "wb") as f:
    pickle.dump(x_test, f)

with open("models/y_test.pkl", "wb") as f:
    pickle.dump(y_test, f)
```

And then recalling it in my test.py file rather than recalling the whole database. I retrained my model, and it correctly saved these models

```
x_test.pkl
y_test.pkl
```

I then reran my test function, and the accuracy did drop, but only by a little bit:

```
Overall testing accuracy: 95.21%
```

This value was a more reliable prediction as it only tested the model on unseen data. Now the only thing about my results that I'm a little worried about is the fact that the confidence score is almost universally 100%. In fact, after testing

using ctrl+f, I realised that everything had a confidence score of 100%, even if it was wrong.

I suspect this has something to do with the hyperparameter tuning that I added, maybe that it was using a too small value for k.

To investigate this, I wrote a print statement to see what parameters it was using and ran the program.

As I suspected, it returned this:

```
Best parameters: {'n_neighbors': 1, 'weights': 'uniform'}
```

This explains the abnormally high confidence scores. I think since my data is too clean, it will lead to exact matches.

I tried adjusting my list so that the minimum no. of neighbours was 59 and ran the program again.

```
final predicton:I
confidence score:51%
final predicton:O
final predicton:B
confidence score:67%
final predicton:minus
confidence score:28%
final predicton:I
confidence score:51%
final predicton:O
confidence score:67%
```

While the confidence scores were more varied, the overall accuracy took a big hit:

```
Overall testing accuracy: 82.84%
```

I don't believe that K=1 has the highest accuracy, so I decided to see if I could check the accuracy of each value for neighbors. After asking ChatGPT, I realised

that I could save the results from the grid search into a csv file. I decided to do this and saw this result:

aram_n_neighbors	param_weights	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
1	uniform	{'n_neighbors': 1, 'weights': 'uniform'}	0.93378119	0.935700576	0.941458733	0.932756964	0.926032661	0.933946025	0.004971551	1
1	distance	{'n_neighbors': 1, 'weights': 'distance'}	0.93378119	0.935700576	0.941458733	0.932756964	0.926032661	0.933946025	0.004971551	1
3	distance	{'n_neighbors': 3, 'weights': 'distance'}	0.928982726	0.928982726	0.935700576	0.935638809	0.917387128	0.929338393	0.006682191	3
5	distance	{'n_neighbors': 5, 'weights': 'distance'}	0.916506718	0.901151631	0.923224568	0.920268972	0.912584054	0.914747189	0.007680759	4
3	uniform	{'n_neighbors': 3, 'weights': 'uniform'}	0.911708253	0.906909789	0.916506718	0.920268972	0.898174832	0.910713713	0.0077142	5
7	distance	{'n_neighbors': 7, 'weights': 'distance'}	0.909788868	0.89059501	0.912667946	0.913544669	0.908741595	0.907067617	0.008424608	6
5	uniform	{'n_neighbors': 5, 'weights': 'uniform'}	0.901151631	0.89059501	0.905950096	0.906820365	0.892411143	0.899385649	0.006743956	7
9	distance	{'n_neighbors': 9, 'weights': 'distance'}	0.893474088	0.881957774	0.901151631	0.912584054	0.899135447	0.897660599	0.010011769	8
7	uniform	{'n_neighbors': 7, 'weights': 'uniform'}	0.89059501	0.876199616	0.892514395	0.902017291	0.893371758	0.890939614	0.00834861	9
11	distance	{'n_neighbors': 11, 'weights': 'distance'}	0.884836852	0.875239923	0.896353167	0.902017291	0.891450528	0.889979552	0.00928868	10

(*Pretty hard to see, but this image shows the number of neighbours in the first column and its accuracy ranking in the second score)*

When the number of neighbours was one, the model had the highest accuracy.

What's weird is that once I implemented the hyperparameter tuning, the accuracy for my previous k values dropped significantly, despite not changing the training data at all.

This will definitely need fixing, but I need to complete other aspects of my program, so I will come back to this later.

Main Menu

One thing that I have forgotten to implement is a main menu, as I have been manually running the test.py and train.py files each time.

All this needs to do is give the users a choice between training mode and testing mode, and then calling the relevant file (e.g. train.py if training mode is selected). This was pretty simple, as all I needed to do was assign a number to training mode and testing mode, and if the number was pressed, it would run the relevant file.

I ran the file and received this message:

```
=== OCR Main Menu ===
1. Train Model
2. Test Model
Enter your choice (1/2):
```

I tried selecting the train and then the test and they both successfully worked. However, I realised that there was no way to leave, so I added an exit button as well, which would break if pressed.

```
=== OCR Main Menu ===
1. Train Model
2. Test Model
3. Exit
Enter your choice (1/2/3):
```

I also made sure that an error message is displayed if an invalid number is pressed:

```
Enter your choice (1/2/3): 4
Invalid choice. Please try again.
```

I think this is all I need for the main menu.

However, another thing I need to add is for testing mode, my program needs to get an input of a file path for a folder of images, and test those images.

Since I was going to be separating the training and testing data, this meant that I could delete the train_test_spklit function as well as this:

```
with open("models/x_test.pkl", "rb") as f:
    x_test = pickle.load(f)

with open("models/y_test.pkl", "rb") as f:
    y_test = pickle.load(f)
```

This works out well as I didn't fully understand this in the first place.

Now, what I need to do is ask the user to enter a file path. This was a pretty straightforward step.

Next, I needed to get all the files from the folder, preprocess them, and then append the image data to a list, which is what will be used for predictions. To do this, I used a similar logic as in my [preprocess.py]() file, where it checks if it uses os.listdir to get all the files and checks if they are a png. It then calls the preprocess function and then appends it to an empty list called x_test.

```
Enter the path to the folder with test images:
```

Since the only folder I currently have is the one I used for training, I tried giving it that file path and it successfully worked.

The only downside is that now I can't see my accuracy score, as there is nothing to compare my labels to. Even though this isn't needed for the project, I wanted to keep it so I could see the true accuracy as I still have to modify my knn parameters.

Another thing that I need to change is how my testing data is output, as Ms. Downie said that it needs to be ‘filename ____ final prediction ____ confidence score’ in the same line. To do this, I created a list for the file names and appended the names of all files from the input folder. Then, I simply adjusted the print statement at the end.

```
File Name: SourceSerif4_bold_32_rot-20_A.png   Final Predicton:A   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_ampersand.png   Final Predicton:ampersand   Confidence Score:
File Name: SourceSerif4_bold_32_rot-20_asterisk.png   Final Predicton:asterisk   Confidence Score: 10
File Name: SourceSerif4_bold_32_rot-20_at.png   Final Predicton:at   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_B.png   Final Predicton:B   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_C.png   Final Predicton:C   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_D.png   Final Predicton:D   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_dollar.png   Final Predicton:dollar   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_E.png   Final Predicton:E   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_exclmark.png   Final Predicton:exclmark   Confidence Score: 10
File Name: SourceSerif4_bold_32_rot-20_F.png   Final Predicton:F   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_G.png   Final Predicton:G   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_greaterthan.png   Final Predicton:greaterthan   Confidence Sco
File Name: SourceSerif4_bold_32_rot-20_H.png   Final Predicton:H   Confidence Score: 100
File Name: SourceSerif4_bold_32_rot-20_hash.png   Final Predicton:hash   Confidence Score: 100
```

This was the output. While this was functional, I didn’t like how it looked

```
File Name: lora_medium_32_rot0_3.png
Final Predicton:3
Confidence Score: 100


File Name: lora_medium_32_rot0_4.png
Final Predicton:4
Confidence Score: 100


File Name: lora_medium_32_rot0_5.png
Final Predicton:5
Confidence Score: 100
```

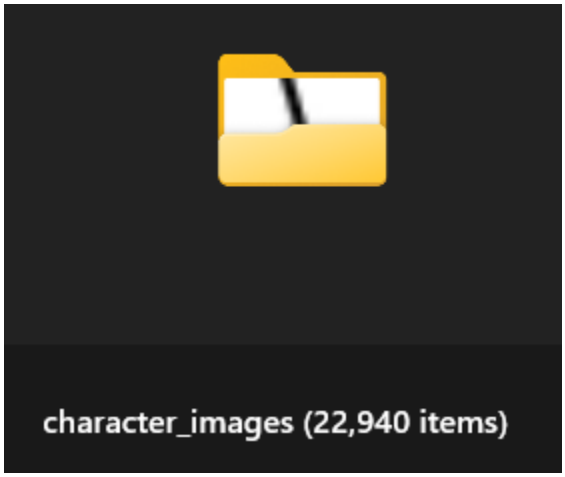
By adding some line spaces, I thought this was much better.

Fixing Training Data

Another issue that I had become aware of was that since file names are not case sensitive, the provided character generation code replaced capital letters with lowercase letters, rather than creating both. This means that my program has only been trained on lowercase letters.

One of my classmates shared with me an edited version of the character generation file which saved the image files by their unicode values, avoiding this issue. I also added rotation to this file, using randomness for better generalisation rather than fixed values of increments of 10. Since this is not part of the project, I figured I could implement this entirely through AI.

I now had almost 23,000 images:



Since my files are now saved in a new format, I need to update my preprocess.py file to get the correct label rather than just the unicode. To do this, I need to take the label string, and convert it from a unicode integer to a character. To do this, I used the chr() function (Python chr() Function). All I needed to do was convert the value to a string and then use this function which gets the corresponding character for the string.

KNN parameter tuning

Now, I want to revisit my KNN parameter tuning. I have realised that the reason for k=1 is because the hyperparameters are being tested on the training data, meaning that there are exact copies from the training, resulting in exact matches. I couldn't understand how to fix this issue, so I decided to scrap this and return back to using the square root of the total number of files as my neighbour values, as I didn't have much time left.

Since I had 23,000 files, my value for k was 151.

I used the image files we were provided to test the accuracy of this model. As they were completely different fonts, I thought this would be a good test.

```
File Name: Tinos_regular_32_T.png
Final Predicton:t
Confidence Score: 58%


File Name: Tinos_regular_32_U.png
Final Predicton:u
Confidence Score: 55%


File Name: Tinos_regular_32_V.png
Final Predicton:v
Confidence Score: 72%


File Name: Tinos_regular_32_W.png
Final Predicton:w
Confidence Score: 66%


File Name: Tinos_regular_32_X.png
Final Predicton:x
Confidence Score: 71%


File Name: Tinos_regular_32_Y.png
Final Predicton:Y
Confidence Score: 30%


File Name: Tinos_regular_32_Z.png
Final Predicton:z
Confidence Score: 83%
```

From the looks of it, the model seemed to guess the correct character pretty frequently, despite not being trained on this font. The confidence score also wasn't absurdly high, which I like, as it means that it is no longer overfitting.

- **Create an installation guide, which identifies:**
    - **The libraries (and any other tools) which must be installed, including a download link (if applicable)**
    - **The Python file that contains main()**
    - **The specific fonts that your model was trained on**

System Requirements:

- Python: Version 3.8 or higher is recommended

Required Libraries:

All of the necessary python libraries can be installed using 'pip' in the terminal/command prompt.

- NumPy

```
None
pip install numpy
```

- Pillow

```
None
pip install pillow
```

Scikit-learn

```
None
pip install scikit-learn
```

How to Run:

The main file that runs the program in main.py. When this file is executed, it will present a menu which can be used to navigate to either testing mode or training mode. The program will automatically have a saved knn model, which can be found in the 'models' folder. If the program is trained again, it will replace this model.

Trained Fonts:

A folder with all used fonts can be found inside the project folder. Otherwise, they can all be found on Google Fonts.

The fonts below are sorted by the type of font, and then their font family:

Serif Fonts

- Lora
    - Lora-Bold
    - Lora-Italic
    - Lora-Medium
    - Lora-Regular
    - Lora-SemiBold

- SourceSerif4
    - SourceSerif4-Bold
    - SourceSerif4-ExtraLight
    - SourceSerif4-Italic
    - SourceSerif4-Light
    - SourceSerif4-Regular
    - SourceSerif4-SemiBold

Sans-Serif Fonts

- OpenSans
    - OpenSans-Bold
    - OpenSans-Italic
    - OpenSans-Light
    - OpenSans-Medium
    - OpenSans-Regular
    - OpenSans-SemiBold

- Roboto
    - Roboto-Bold
    - Roboto-Italic
    - Roboto-Light
    - Roboto-Medium
    - Roboto-Regular
    - Roboto-Thin

Monospace Fonts

- RobotoMono
    - RobotoMono-Bold
    - RobotoMono-Italic
    - RobotoMono-Light
    - RobotoMono-Medium
    - RobotoMono-Regular
    - RobotoMono-Thin

'Handwritten' Fonts

- Handlee-Regular
- IndieFlower-Regular

Testing and Evaluating

- **Summarise the results of the testing, including:**
    - **The amount of labelled data used to train the model**
    - **Percentage of accurate results**
    - **Average confidence level for accurate and inaccurate results**
    - **The characters with the highest and lowest rates of accuracy**
    - **The effect of font and style on accuracy**
    - **Any other interesting result**

This model was trained on over 23000 files of labelled training data. For my test report, I used 2249 unseen files using the same fonts and styles, but different rotations, to get more reliable results.

The overall accuracy of this model was 91.15%. The average confidence score when the model was correct was 65.95%, however the average confidence score when the model was incorrect was 33.95%. This is a good indicator as this means that the model isn't overfitting, increasing its reliability.

Characters which recorded the highest accuracy (100%)::

- o, s, u, +, -, 0, 9, @, C, c

Characters which recorded the lowest accuracies:

- J (67.7%) -> most commonly confused with '-' (8% of the time)
- A (64.5%) -> most commonly confused with '4' (10% of the time)
- $ (58.1%) -> most commonly confused with 's' (31% of the time)

These results were interesting as while some of them were expected ('$' being mixed up with 's', 'A' being mixed up with '4'), others were surprising ('0' and 'o' having 100% accuracy while looking similar, 'J' being mixed up with '-').

I also noticed that '-' was the most common guess, appearing 101 times in my test report, despite there only being 31 instances in my test data.

The most accurate font/styles (98.6% accuracy for each, one incorrect output):

- Lora Bold
- Lora Medium
- Lora Semi-Bold
- OpenSans Medium
- OpenSans SemiBold
- Roboto Italic
- SourceSerif4 Regular
- SourceSerif4 SemiBold

The least accurate font/styles:

- IndieFlower Regular (55%)
- Roboto Thin (62%)
- RobotoMono Thin (68%)
- Handlee Regular (73%)
- SourceSerif4 ExtraLight (73%)

From these results, it is evident that the model thrived when testing semi-bold and medium styles, especially for serif and san serif fonts. The model struggled against the 2 handwritten fonts, IndieFlower and Handlee. This can be justified as there is a lot less training data on handwritten fonts. It also performed poorly against thinner/lighter variations of fonts.

Overall, the testing process indicated that the model is overall effective in identifying a range of fonts with different styles and rotations. The model also showed reliability as the confidence score was higher when it was correct, and lower when it was incorrect. The model could be improved by increasing the diversity of handwritten data as well as thinner fonts.

- **Evaluate your final program with reference to your initial functionality requirements. Discuss the impact of the libraries and approaches you selected on the program and the development process.**

My final program successfully met my initial functionality requirements by adopting a modular approach which utilised external libraries. When the program is run, it provides a main menu which allows the user to choose between training and test mode, as my requirements stated. In addition to this, I also added an exit button, so that users can easily exit the program, increasing usability.

When in training mode (train.py), a dataset is correctly loaded from a given file path. After this, it is run through a preprocess function to ensure that the images file type and size is appropriate and that it is converted to grayscale. For this process, I utilised the 'Pillow' library, which significantly simplified image manipulation. Following this, the images were converted into a 1D vector through 'NumPy', which allowed me to convert the image files into the required input for my model.

After this process is completed, this dataset is used to train the K-Nearest Neighbors model, as per the requirements. This model was implemented through the 'Scikit-learn' library, which included a ready-to-use knn model and significantly streamlined the training process. When setting up the model, I attempted to use GridSearchCV, which is an algorithm included in scikit-learn, that tries to find the most optimal parameters. However, I was unsuccessful in properly implementing this, meaning that it didn't contribute to my project. Rather, I found my final parameters after testing different values manually.

After the model was trained, it was then saved to a file as outlined in the functionality requirements. To do this, I utilised the 'pickle' library, which saved the knn model in a .pkl file, allowing the trained model to be saved, so that it could be reloaded later for the testing phase.

In testing mode, the program successfully asks the user for a file path to the images which should be tested, and preproccesses all the images. It also reloads the pkl file of the saved KNN model and predicts the characters using this. It then outputs the final prediction and the confidence score for each file, as outlined in the functionality requirements. There was limited use of libraries for this component.

Overall, my final program successfully met all the outlined my initial program requirements by adopting a modular approach which separated preprocessing, training, and testing into different scripts. The use of external libraries such as scikit-learn, pickle, pillow, and numpy supported the program by simplifying the development process and making it more efficient. Although the final program is fully functional and performed decently, the accuracy could have been improved if I had successfully implemented GridSearchCV.